

Softwarekonstruktion WS 15/16 – Übung 2

Ausgabe: 09.11.2015, Abgabe: 25.11.2015 12:00 Uhr

Präsenzübung

2.1 Petrinetze – Modellierung

Es wird eine Klausur geschrieben, bei der zwei Tutoren für die Aufsicht eingeteilt sind und vorne an der Tafel sitzen. Hat ein Student während der Klausur eine Frage, meldet er sich. Daraufhin geht ein Tutor zu diesem Studenten um die Frage zu beantworten. Ist die Frage beantwortet, kehrt er zur Tafel zurück. Der andere Tutor bleibt währenddessen an der Tafel sitzen.

1. Modellieren Sie dieses Szenario als Petrinetz. Es soll unterschieden werden können, welcher Tutor an der Tafel sitzen bleibt.
2. Erstellen Sie den Erreichbarkeitsgraphen zu Ihrem Petrinetz.
3. Erweitern Sie Ihr Petrinetz, so dass sich die Tutoren mit der Beantwortung von Fragen abwechseln müssen.
4. Erstellen Sie den Erreichbarkeitsgraphen zu Ihrem erweiterten Petrinetz.

2.2 Petrinetze – Eigenschaften von Transitionen

Sei $ST = (S, T, F, W, K, M_0)$ ein S/T-Netz. Erklären Sie die folgenden Definitionen in Worten.

1. Transition $t \in T$ ist *aktivierbar* $\Leftrightarrow \exists M_i \in [M_0]$ mit: $M_i[t]$
2. Transition $t \in T$ ist *lebendig* $\Leftrightarrow \forall M_i \in [M_0]$ gilt: $\exists M_j \in [M_i]$ mit: $M_j[t]$
3. Transition $t \in T$ ist *tot* $\Leftrightarrow \forall M_i \in [M_0]$ gilt: $\neg M_i[t]$

2.3 Petrinetze – Eigenschaften von Petrinetzen

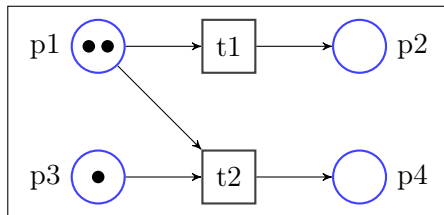
Sei $ST = (S, T, F, W, K, M_0)$ ein S/T-Netz. Erklären Sie die folgenden Definitionen in Worten.

1. Das Petrinetz ST ist *k-beschränkt* $\Leftrightarrow \forall M_1 \in [M_0], \forall s \in S : M_1(s) \leq k$
2. Das Petrinetz ST ist *sicher* $\Leftrightarrow \forall M_1 \in [M_0], \forall s \in S : M_1(s) \leq 1$
3. Das Petrinetz ST ist *deadlockfrei* $\Leftrightarrow \forall M_1 \in [M_0], \exists t \in T : M_1[t]$
4. Das Petrinetz ST ist *lebendig* $\Leftrightarrow \forall M_1 \in [M_0], \forall t \in T : \exists M_2 \in [M_1] : M_2[t]$
5. Das Petrinetz ST ist *tot* $\Leftrightarrow \forall t \in T : \neg M_0[t]$

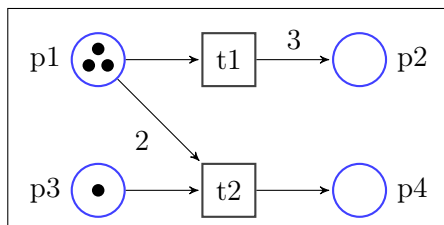
2.4 Petrinetze – Erreichbarkeitsgraph

Zeichnen Sie die Erreichbarkeitsgraphen zu folgenden Graphen. Ist ein Erreichbarkeitsgraph unendlich, markieren Sie dies an entsprechender Stelle mit (...). Geben Sie jeweils an, ob die Petrinetze k -beschränkt (mit welchem kleinsten k), sicher, deadlockfrei, lebendig und/oder tot sind und begründen Sie die Aussagen anhand der Erreichbarkeitsgraphen.

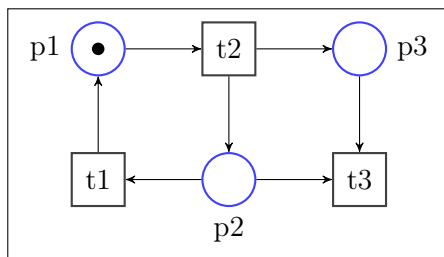
1.



2.



3.



Hausübung

2.5 Petrinetze – Netzeigenschaften (10 Punkte)

Geben Sie für die Petrinetze in Abbildung 1 jeweils an, ob das gesamte Netz *lebendig*, *deadlockfrei* und/oder *tot* ist. Geben Sie zudem für jedes der Netze an, ob es *beschränkt* ist und nennen Sie im positiven Falle das kleinste k , für welches das Netz k -beschränkt ist. Geben Sie schließlich an, welche Netze *sicher* sind. Füllen Sie dazu untenstehende Tabelle aus.

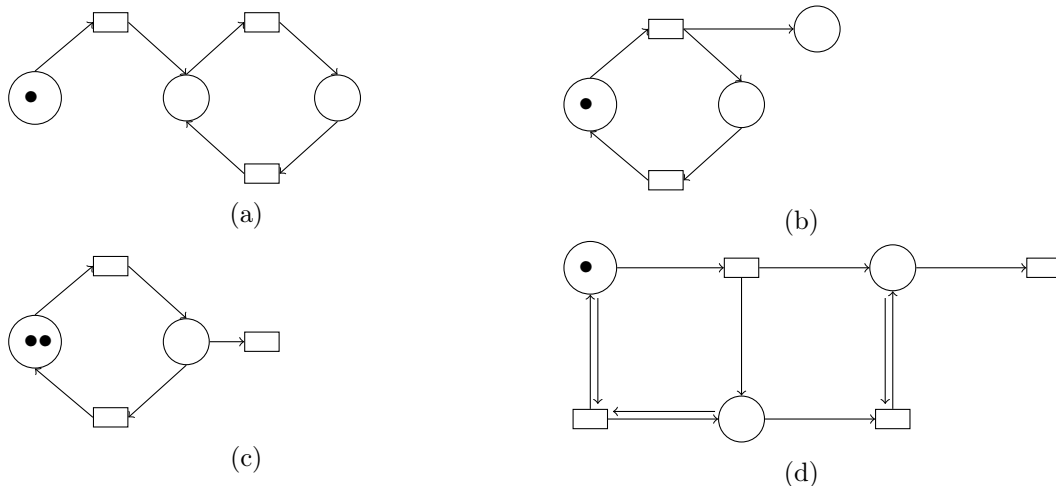


Abbildung 1: Petrinetze zu Aufgabe 2.5

	a)	b)	c)	d)
lebendig				
deadlockfrei				
tot				
k -beschr., $k =$				
sicher				

2.6 Petrinetze – Modellierung und Erreichbarkeit (20 Punkte)

Drei Raucher sitzen an einem Tisch, der zu Beginn leer ist. Jeder von ihnen braucht zum Rauchen die drei Zutaten Tabak, Blättchen und Streichhölzer. Jeder von ihnen hat eine der Zutaten in unendlicher Menge, benötigt also zwei weitere Zutaten, damit er rauchen kann. Alle Raucher verfügen über jeweils eine unterschiedliche Zutat in unendlicher Menge. Es gibt außerdem einen Tabakhändler, der über alle drei Zutaten in unendlicher Menge verfügt. Wenn der Tisch leer ist, legt er immer zwei zufällig ausgewählte Zutaten auf den Tisch. Es gibt dann einen Raucher, der die zwei ihm fehlenden Zutaten vom Tisch nehmen kann um zu rauchen.

1. Modellieren Sie das s.g. Raucherproblem als Petrinetz. Modellieren Sie dabei für jede Zutat eine eigene Stelle.
2. Erstellen Sie den Erreichbarkeitsgraphen zu Ihrem Petrinetz aus Teilaufgabe 1
3. Ist Ihr Petrinetz aus Teilaufgabe 1:

- a) k-beschränkt,
- b) sicher,
- c) lebendig,
- d) deadlockfrei,
- e) tot?

Begründen Sie Ihre Aussagen mit möglichst wenigen Sätzen!

2.7 EMF (10 Punkte)

Ein Entwickler hatte den Auftrag mittels des *Eclipse Modeling Frameworks (EMF)* ein Szenario rund um eine Playlist sowohl mittels des EMF-Diagrammeditors grafisch zu modellieren, als auch die entsprechenden annotierten Java-Interfaces mittels EMF generieren zu lassen.

Leider arbeitete der Entwickler sehr schludrig und achtlos. Nach seinem Weggang von der Firma sind nur noch ein *nicht vollständiger* Teil des Ecore-Klassendiagramms (Abbildung 2) und die in (Abbildung 3) dargestellten *annotierten* Java-Interfaces vorhanden.

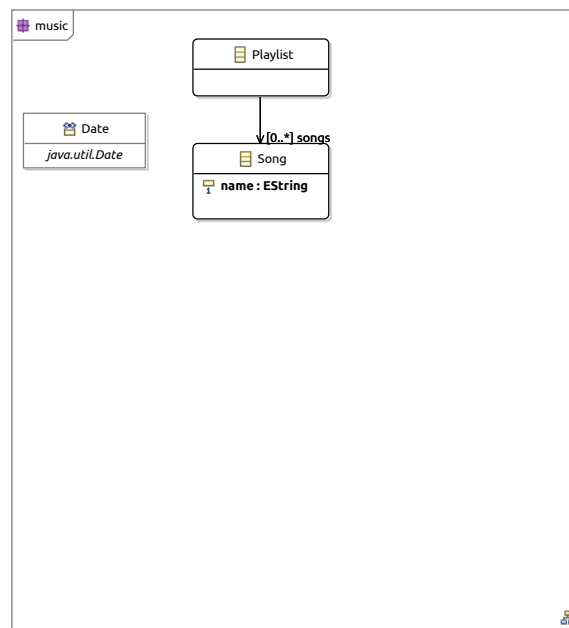


Abbildung 2: Fragment des Ecore-Klassendiagramms (Aufgabe 2.7)

Ihre Aufgabe ist nun, die fehlenden Informationen des Ecore-Klassendiagramms zu rekonstruieren. Ergänzen Sie dazu das Ecore-Klassendiagramm gemäß der gegebenen annotierten Java-Interfaces. Modellieren Sie die fehlenden Klassen, deren Attribute samt Typen, sowie die korrekten Referenzen samt Kardinalitäten. Achten Sie darauf, dass **Kuenstler** eine Generalisierung von **SingleKuenstler** sowie **Band** ist. Die Klassen **EList** und **EObject** sind Klassen des EMF-Frameworks und müssen nicht modelliert werden. Getter- und Setter-Methoden müssen ebenfalls nicht modelliert werden.

```
package music;
import org.eclipse.emf.common.util.EList;

/**
 * @model
 */
public interface Playlist extends EObject {
    /**
     * @return the value of the '<em>Songs</em>' reference list.
     * @model type="music.Song"
     */
    EList getSongs();

    /**
     * @return the value of the '<em>Name</em>' attribute.
     * @model required="true"
     */
    String getName();

    /**
     * @param value the new value of the '<em>Name</em>' attribute.
     */
    void setName(String value);
} // Playlist

package music;
import java.util.Date;

/**
 * @model
 */
public interface Song extends EObject {
    /**
     * @return the value of the '<em>Name</em>' attribute.
     * @model required="true"
     */
    String getName();

    /**
     * @param value the new value of the '<em>Name</em>' attribute.
     */
    void setName(String value);

    /**
     * @return the value of the '<em>Year</em>' attribute.
     * @model dataType="music.Date" required="true"
     */
    Date getYear();

    /**
     * @param value the new value of the '<em>Year</em>' attribute.
     */
    void setYear(Date value);

    /**
     * @return the value of the '<em>Interpret</em>' reference.
     * @model opposite="songs" required="true"
     */
    Kuenstler getInterpret();

    /**
     * @param value the new value of the '<em>Interpret</em>' reference.
     */
    void setInterpret(Kuenstler value);
} // Song

package music;
import org.eclipse.emf.common.util.EList;

/**
 * @model abstract="true"
 */
public interface Kuenstler extends EObject {
    /**
     * @return the value of the '<em>Name</em>' attribute.
     * @model required="true"
     */
    String getName();

    /**
     * @param value the new value of the '<em>Name</em>' attribute.
     */
    void setName(String value);

    /**
     * @return the value of the '<em>Songs</em>' reference list.
     * @model type="music.Song" opposite="interpret" required="true"
     */
    EList getSongs();
} // Kuenstler

package music;
import java.util.Date;

/**
 * @model
 */
public interface SingleKuenstler extends Kuenstler {
    /**
     * @return the value of the '<em>Birthday</em>' attribute.
     * @model dataType="music.Date" required="true"
     */
    Date getBirthday();

    /**
     * @param value the new value of the '<em>Birthday</em>' attribute.
     */
    void setBirthday(Date value);
} // SingleKuenstler

package music;
import org.eclipse.emf.common.util.EList;

/**
 * @model
 */
public interface Band extends Kuenstler {
    /**
     * @return the value of the '<em>Mitglieder</em>' reference list.
     * @model type="music.SingleKuenstler" lower="2"
     */
    EList getMitglieder();
} // Band
```

Abbildung 3: Annotierte Java-Interfaces (Aufgabe 2.7)